

Addis Ababa Science and Technology University

Artificial Intelligence & Robotics Center of Excellence (AI&R CoE)

Machine Learning Lab Manual for MSc. AI Students

Course Code: ARIN6012

Author: Habtamu Girum (MSc.)

Editor 1: Habib Mohammed (PhD.)

Signature: _____

Date: _____

Editor 2: Ewnetu Abebe (MSc.)

Signature: _____

Date: _____

Approved By: Habib Mohammed (PhD.)

Signature: _____

Date: _____

Table of Contents

Learning Outcomes **3**

1 Advanced Regression Models **4**
- 1.1 Regression revision . . . 4
- 1.2 Best Practices for Evaluating Regression Performance . . . 5
- 1.3 Bias and Variance Analysis . . . 6

2 Advanced Classification Models **9**
- 2.1 Naïve Bayes Classification . . . 9
- 2.2 Linear Discriminant Analysis (LDA) . . . 9
- 2.3 Support Vector Machines (SVM) and Kernel Functions . . . 10

3 Ensembles and Neural Networks **14**
- 3.1 Overview . . . 14
- 3.2 Ensemble Learning . . . 14
- 3.3 Neural Networks . . . 15
- 3.4 Classic Neural Network Topologies . . . 15
- 3.5 Activation Functions . . . 15
- 3.6 Forward and Back-Propagation . . . 16
- 3.7 Optimization Algorithms . . . 16
- 3.8 Key Parameters of Neural Networks . . . 16

4 Deep Neural Networks I **20**
- 4.1 Overview . . . 20
- 4.2 Neural Networks and Classic Topologies . . . 20
- 4.3 Activation Functions . . . 21
- 4.4 Forward and Back-Propagation . . . 21
- 4.5 Optimization Algorithms . . . 22
- 4.6 Key Parameters of Neural Networks . . . 22
- 4.7 Advanced Training Techniques . . . 22

5 Convolutional Neural Networks **27**
- 5.1 Overview . . . 27
- 5.2 CNN Architecture . . . 27
- 5.3 Convolution Operation . . . 28
- 5.4 Stacking Layers for Deep CNNs . . . 28
- 5.5 Multi-Class Image Classification . . . 28
- 5.6 Object Detection . . . 29
- 5.7 Deep Residual Learning . . . 29

6 Recurrent Neural Networks **33**
6.1 Overview . . . 33
6.2 Basic Recurrent Unit (Elman Network) . . . 33
6.3 Training RNNs . . . 33
6.4 Advanced Recurrent Units . . . 34
6.5 Vanishing Gradient Problem . . . 34

7 Transformers and Deep Generative Models **38**
7.1 Transformers . . . 38
7.2 Autoencoders and Deep Generative Models . . . 40

8 Active Learning, Transfer Learning, and Applications of Deep Learning **43**
8.1 Active Learning (AL) . . . 43
8.2 Sampling Techniques in Active Learning . . . 43
8.3 Transfer Learning (TL) . . . 44
8.4 Transfer Learning Strategies . . . 44
8.5 Practical Applications of AL and TL . . . 44
8.6 Applications of Deep Learning Models . . . 45

Learning Outcomes

Upon successful completion of this course, students will be able to:

- **Apply** advanced mathematical, statistical, and computational foundations to machine learning problems.
- **Design and implement** supervised, unsupervised, and deep learning models using appropriate algorithms and tools.
- **Analyze and evaluate** the performance, robustness, and generalization ability of machine learning models.
- **Develop and conduct** independent experiments, including data preprocessing, model training, hyperparameter tuning, and validation.
- **Identify and address** issues related to bias, fairness, interpretability, and ethical considerations in machine learning systems.

Chapter 1

Advanced Regression Models

1.1 Regression revision

Regression: Quick Revision

Regression is a supervised learning technique used to model the relationship between one or more independent variables (features) and a continuous dependent variable (target).

- **Common Regression Types**
  - **Simple Linear Regression:** One input variable and one output variable.
$$y = \beta_0 + \beta_1 x + \varepsilon \tag{1.1}$$
  - **Multiple Linear Regression:** Multiple input variables.
$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon \tag{1.2}$$
- **Key Assumptions of Linear Regression**
  - Linearity between input variables and the output variable
  - Independence of errors
  - Homoscedasticity (constant variance of errors)
  - Normal distribution of errors
  - No multicollinearity among predictor variables

1.2 Best Practices for Evaluating Regression Performance

Model Evaluation Techniques

Train–Test Split

- Split the dataset into training and testing sets (e.g., 80% / 20%).
- Prevents evaluating the model on data it has already seen.

Cross-Validation

- K-Fold Cross-Validation improves reliability.
- Reduces dependency on a single data split.

Common Evaluation Metrics

- **Mean Absolute Error (MAE):** Measures the average absolute error.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{1.3}$$

- **Mean Squared Error (MSE):** Penalizes large errors.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{1.4}$$

- **Root Mean Squared Error (RMSE):** Has the same unit as the target variable.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1.5}$$

- R^2 **(Coefficient of Determination):** Measures how much variance is explained by the model.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{1.6}$$

Best Practices

- Use multiple evaluation metrics, not just one.
- Always evaluate model performance on unseen data.
- Compare model performance against a baseline model.

1.3 Bias and Variance Analysis

Bias and variance are two fundamental sources of error in machine learning models. Understanding their tradeoff helps in selecting models that generalize well to unseen data.

Bias

- Error due to oversimplified models
- Model fails to capture underlying patterns
- Leads to **underfitting**

Variance

- Error due to overly complex models
- Model fits noise in training data
- Leads to **overfitting**

Bias–Variance Tradeoff

- Increasing model complexity:
  - Decreases bias
  - Increases variance
- Goal: Balance bias and variance to minimize total error

Diagnosis

- High training error and high testing error $\rightarrow$ High bias
- Low training error and high testing error $\rightarrow$ High variance

Mathematical Insight

The expected prediction error can be decomposed as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \tag{1.7}$$

Applications in Machine Learning

- Helps in selecting appropriate model complexity
- Guides the use of regularization techniques
- Improves model generalization on unseen data

Dataset

Dataset

Dataset: project uses the MAGIC Gamma Telescope dataset obtained from the UCI Machine Learning Repository
Dataset Link: https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope
Dataset Description: The dataset is generated using Monte Carlo simulations to model the detection of high-energy gamma particles by a ground-based atmospheric Cherenkov telescope.

Lab Tasks

Task 3: Implement a Simple Machine Learning Model (Logistic Regression)

In this task, a Logistic Regression classifier is implemented to perform binary classification. The dataset is preprocessed, split into training and testing sets, and the model performance is evaluated.

- Split the dataset into training (80%) and testing (20%) sets.
- Preprocess the data using feature scaling and class balancing.
- Train a Logistic Regression classifier.
- Predict the test set and evaluate the model using accuracy and classification metrics.

Sample Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from imblearn.over_sampling import RandomOverSampler

# Load dataset
cols = [
    "fLength", "fWidth", "fSize", "fConc", "fConc1",
    "fAsym", "fM3Long", "fM3Trans", "fAlpha", "fDist", "class"
]
df = pd.read_csv("magic04.data", names=cols)

# Encode target variable
df["class"] = (df["class"] == "g").astype(int)

# Split dataset
```

```
train, valid, test = np.split(
    df.sample(frac=1, random_state=42),
    [int(0.6 * len(df)), int(0.8 * len(df))]
)

# Scaling and oversampling function
def scale_dataset(dataframe, oversample=False):
    X = dataframe.iloc[:, :-1].values
    y = dataframe.iloc[:, -1].values

    scaler = StandardScaler()
    X = scaler.fit_transform(X)

    if oversample:
        ros = RandomOverSampler()
        X, y = ros.fit_resample(X, y)

    return X, y

X_train, y_train = scale_dataset(train, oversample=True)
X_valid, y_valid = scale_dataset(valid)
X_test, y_test = scale_dataset(test)

# Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Prediction and evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

Expected Learning Outcomes

- Students will be able to load, preprocess, and explore a real-world dataset for binary classification.
- Students will understand the role of feature scaling and class balancing in improving model performance.
- Students will gain hands-on experience implementing a Logistic Regression classifier using `scikit-learn`.
- Students will be able to evaluate classification performance using accuracy, precision, recall, and F1-score.
- Students will develop the ability to interpret model results and recognize limitations such as overfitting and class imbalance.

Chapter 2

Advanced Classification Models

2.1 Naïve Bayes Classification

- A **generative probabilistic classifier** based on Bayes' theorem. - Assumes **conditional independence** among features given the class label.
Bayes' Theorem:

$$P(Y|X) = \frac{P(X|Y)P(Y)}{P(X)}$$

Naïve Bayes Model:

$$P(Y|X) \propto P(Y) \prod_{i=1}^{n} P(x_i|Y)$$

Applications: Text classification, spam filtering, medical diagnosis.

2.2 Linear Discriminant Analysis (LDA)

- A **generative classification technique** assuming Gaussian class distributions.
- Finds a **linear combination of features** that best separates classes.
Discriminant Function:

$$\delta_k(X) = X^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log P(Y = k)$$

Key Assumptions: - Normal distribution of features - Equal covariance matrices across classes
Applications: Pattern recognition, face recognition, medical classification.

2.3 Support Vector Machines (SVM) and Kernel Functions

SVM

Decision function:

$$f(x) = w^T x + b$$

Hard margin optimization:

$$\min_{w,b} \frac{1}{2}||w||^2 \quad \text{s.t. } y_i(w^T x_i + b) \geq 1$$

Soft margin:

$$\min_{w,b,\xi} \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i, \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

Kernel Functions

- **Linear:** $K(x, x') = x^T x'$
- **Polynomial:** $K(x, x') = (x^T x' + c)^d$
- **RBF/Gaussian:** $K(x, x') = \exp\left(-\frac{||x-x'||^2}{2\sigma^2}\right)$

Lab Exercise: Advanced Classification Models

Lab Objectives

After completing this lab, students will be able to:

- Understand probabilistic and margin-based classification models.
- Implement Gaussian Naïve Bayes for classification.
- Implement Logistic Regression as a discriminative classifier.
- Implement Support Vector Machine (SVM) for classification.
- Evaluate model performance using classification metrics.

Dataset

Dataset

Dataset: project uses the MAGIC Gamma Telescope dataset obtained from the UCI Machine Learning Repository
Dataset Link: https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope
Dataset Description: The dataset is generated using Monte Carlo simulations to model the detection of high-energy gamma particles by a ground-based atmospheric Cherenkov telescope.

Lab Steps

1. Gaussian Naïve Bayes Classification

Gaussian Naïve Bayes is a **generative probabilistic model** that assumes features follow a normal distribution and are conditionally independent given the class label.
Student Implementation:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report

nb_model = GaussianNB()
nb_model = nb_model.fit(X_train, y_train)

y_pred = nb_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

2. Logistic Regression

Logistic Regression is a **discriminative classification model** that directly models the conditional probability $P(Y|X)$. It uses a sigmoid function to map predictions to probabilities and works well for linearly separable data.
Student Implementation:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

lg_model = LogisticRegression()
lg_model = lg_model.fit(X_train, y_train)

y_pred = lg_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

3. Support Vector Machine (SVM)

Support Vector Machine is a **margin-based classifier** that finds an optimal hyperplane separating classes. It is effective for both linear and non-linear classification problems.
Student Implementation:

```
from sklearn.svm import SVC
from sklearn.metrics import classification_report

svm_model = SVC()
svm_model = svm_model.fit(X_train, y_train)

y_pred = svm_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

Expected Outcomes

- Successfully implement Naïve Bayes, Logistic Regression, and SVM classifiers.
- Compare performance using precision, recall, F1-score, and accuracy.
- Understand differences between probabilistic, discriminative, and margin-based models.
- Gain hands-on experience with real-world classification workflows.

Chapter 3

Ensembles and Neural Networks

3.1 Overview

This chapter introduces ensemble learning methods and neural networks. Students will learn how multiple models can be combined to improve performance and how neural networks learn complex patterns through forward and back-propagation.

3.2 Ensemble Learning

Ensemble learning combines multiple models to produce a stronger classifier or regressor.
Key Idea: A group of weak learners can form a strong learner when combined properly.
Common Ensemble Techniques:

- **Bagging (Bootstrap Aggregation)** – reduces variance (e.g., Random Forest).
- **Boosting** – focuses on hard-to-classify samples (e.g., AdaBoost, Gradient Boosting).
- **Voting** – combines predictions from multiple models.

Advantages:

- Higher accuracy
- Better generalization
- Reduced overfitting

3.3 Neural Networks

A neural network is a computational model inspired by the human brain, consisting of interconnected neurons arranged in layers.

Basic Structure:

- Input Layer
- Hidden Layer(s)
- Output Layer

Neuron Computation:

$$z = w^T x + b$$

$$a = f(z)$$

where $f(\cdot)$ is an activation function.

3.4 Classic Neural Network Topologies

- **Single-Layer Perceptron** – Linear classifier
- **Multi-Layer Perceptron (MLP)** – Non-linear classifier
- **Feedforward Neural Network** – Data flows in one direction

3.5 Activation Functions

Activation functions introduce non-linearity into the network.

- **Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **ReLU:**

$$f(x) = \max(0, x)$$

- **Softmax (for multi-class):**

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

3.6 Forward and Back-Propagation

Forward Propagation:

- Input data is passed through layers.
- Output is computed using weights and activation functions.

Back-Propagation:

- Error is propagated backward.
- Weights are updated using gradients.

Loss Function (Example – Cross Entropy):

$$L = -\sum y \log(\hat{y})$$

3.7 Optimization Algorithms

Gradient Descent (GD):

$$w = w - \eta \nabla L(w)$$

Stochastic Gradient Descent (SGD):

- Updates weights using one sample at a time.
- Faster and scalable for large datasets.

3.8 Key Parameters of Neural Networks

- Learning rate
- Number of hidden layers
- Number of neurons per layer
- Activation function
- Batch size
- Number of epochs

Lab Exercise: Neural Networks and Ensembles

Lab Objectives

After completing this lab, students will be able to:

- Implement a Multi-Layer Perceptron (MLP).
- Understand the effect of activation functions.
- Train neural networks using gradient-based optimization.
- Implement a simple ensemble classifier.

Lab Steps

1. Multi-Layer Perceptron (MLP)

Student Implementation:

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

mlp = MLPClassifier(hidden_layer_sizes=(50,),
                    activation='relu',
                    solver='adam',
                    max_iter=500,
                    random_state=42)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print(classification_report(y_test, y_pred))
```

2. Ensemble Classifier (Voting)

Student Implementation:

```
from sklearn.ensemble import VotingClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

nb = GaussianNB()
lr = LogisticRegression()
svm = SVC(probability=True)

ensemble = VotingClassifier(
    estimators=[('nb', nb), ('lr', lr), ('svm', svm)],
    voting='soft'
)

ensemble.fit(X_train, y_train)
y_pred = ensemble.predict(X_test)

print(classification_report(y_test, y_pred))
```

Expected Outcomes

- Understand neural network architecture and learning process.
- Implement MLP with different activation functions.
- Observe performance improvement using ensemble methods.
- Gain practical experience with modern machine learning models.

Chapter 4

Deep Neural Networks I

4.1 Overview

This chapter introduces the foundations of deep neural networks, including network architecture, training mechanisms, regularization techniques, and optimization strategies used to build robust and scalable deep learning models.

4.2 Neural Networks and Classic Topologies

A neural network is composed of layers of interconnected neurons that learn representations from data.

Classic Topologies:

- **Single-Layer Perceptron** – Linear decision boundary
- **Multi-Layer Perceptron (MLP)** – One or more hidden layers
- **Deep Feedforward Network** – Multiple hidden layers

4.3 Activation Functions

Activation functions introduce non-linearity into neural networks.

- **Sigmoid:**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **Tanh:**

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

- **ReLU:**

$$f(x) = \max(0, x)$$

- **Softmax (Output Layer):**

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

4.4 Forward and Back-Propagation

Forward Propagation:

- Input data flows through the network layer by layer.
- Output predictions are computed using weights and activation functions.

Back-Propagation:

- Computes gradients of the loss function with respect to weights.
- Uses the chain rule to propagate error backward.

Loss Function (Cross-Entropy):

$$L = -\sum y \log(\hat{y})$$

4.5 Optimization Algorithms

Gradient Descent (GD):

$$w = w - \eta \nabla L(w)$$

Stochastic Gradient Descent (SGD):

- Updates parameters using mini-batches.
- Faster convergence on large datasets.

4.6 Key Parameters of Neural Networks

- Learning rate
- Number of hidden layers
- Neurons per layer
- Activation functions
- Batch size
- Epochs

4.7 Advanced Training Techniques

Weight Initialization

Proper initialization helps prevent vanishing and exploding gradients.

- **Xavier Initialization** – for sigmoid/tanh
- **He Initialization** – for ReLU

Regularization Techniques

L2 Regularization:

$$L_{reg} = L + \lambda \sum w^2$$

Dropout:

- Randomly disables neurons during training.
- Reduces overfitting.

Batch and Layer Normalization

- **Batch Normalization** – normalizes mini-batch activations.
- **Layer Normalization** – normalizes across features.

Gradient Checking

Gradient checking verifies correctness of back-propagation by comparing analytical and numerical gradients.

Convergence and Learning Rate Scheduling

- Step decay
- Exponential decay
- Adaptive optimizers (Adam, RMSProp)

Hyperparameter Tuning

- Grid Search
- Random Search
- Cross-validation

Lab Exercise: Deep Neural Networks I

Lab Objectives

After completing this lab, students will be able to:

- Build and train a Multi-Layer Perceptron.
- Apply regularization and normalization techniques.
- Tune hyperparameters for better performance.

Lab Steps

1. Basic MLP Implementation

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report

mlp = MLPClassifier(hidden_layer_sizes=(64, 32),
                    activation='relu',
                    solver='adam',
                    max_iter=500,
                    random_state=42)

mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)

print(classification_report(y_test, y_pred))
```

2. Regularization and Dropout (L2)

```
mlp_reg = MLPClassifier(hidden_layer_sizes=(64, 32),
                        activation='relu',
                        alpha=0.001,
                        max_iter=500,
                        random_state=42)

mlp_reg.fit(X_train, y_train)
y_pred = mlp_reg.predict(X_test)

print(classification_report(y_test, y_pred))
```

3. Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'hidden_layer_sizes': [(64,), (64, 32)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001]
}

grid = GridSearchCV(MLPClassifier(max_iter=500),
                    param_grid,
                    cv=3)

grid.fit(X_train, y_train)
print("Best Parameters:", grid.best_params_)
```

Expected Outcomes

- Understand deep neural network training mechanisms.
- Apply regularization to reduce overfitting.
- Improve model convergence using tuning strategies.
- Gain hands-on experience with deep learning fundamentals.

Chapter 5

Convolutional Neural Networks

5.1 Overview

Convolutional Neural Networks (CNNs) are a specialized class of deep neural networks designed for processing grid-like data such as images. CNNs automatically learn spatial hierarchies of features, making them highly effective for image classification and object detection tasks.

5.2 CNN Architecture

A typical CNN is composed of the following layers:

- **Convolutional Layer** – Extracts local features using filters.
- **Activation Layer** – Introduces non-linearity (ReLU).
- **Pooling Layer** – Reduces spatial dimensions.
- **Fully Connected Layer** – Performs classification.

5.3 Convolution Operation

The convolution operation applies a filter (kernel) over an input image to produce a feature map.

$$(I * K)(x, y) = \sum_m \sum_n I(x + m, y + n)K(m, n)$$

Key Parameters:

- Kernel size
- Stride
- Padding
- Number of filters

5.4 Stacking Layers for Deep CNNs

Deep CNNs are formed by stacking multiple convolutional and pooling layers.
Methodology:

- Early layers capture low-level features (edges, textures).
- Deeper layers capture high-level features (objects, shapes).
- Fully connected layers perform multi-class classification.

Output Layer (Softmax):

$$P(y_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5.5 Multi-Class Image Classification

CNNs are widely used for multi-class image classification tasks.
Loss Function (Categorical Cross-Entropy):

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

Evaluation Metrics: Accuracy, Precision, Recall, F1-score.

5.6 Object Detection

Object Detection Overview

Object detection identifies and localizes objects within an image using bounding boxes and class labels.

Two Categories:

- Two-stage detectors (R-CNN, Faster R-CNN)
- One-stage detectors (YOLO, SSD)

YOLO (You Only Look Once)

YOLO is a one-stage object detection algorithm that treats detection as a regression problem.

Key Characteristics:

- Divides image into grid cells.
- Predicts bounding boxes and class probabilities in one pass.
- Extremely fast and suitable for real-time detection.

5.7 Deep Residual Learning

Residual Networks (ResNet)

Deep Residual Networks address the degradation problem in very deep networks by introducing shortcut connections.

Residual Mapping:

$$H(x) = F(x) + x$$

Advantages:

- Enables very deep networks (50+ layers)
- Improves gradient flow
- Achieves higher accuracy

Lab Exercise: Convolutional Neural Networks

Lab Objectives

After completing this lab, students will be able to:

- Build a CNN for image classification.
- Understand layer stacking in deep CNNs.
- Apply pre-trained CNN models.
- Perform basic object detection.

Lab Steps

1. CNN for Image Classification

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

2. Transfer Learning with ResNet

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

base_model = ResNet50(weights='imagenet', include_top=False,
                      input_shape=(224,224,3))

x = GlobalAveragePooling2D()(base_model.output)
output = Dense(10, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)
```

3. YOLO Object Detection (Conceptual)

Task:

- Load a pre-trained YOLO model.
- Perform object detection on sample images.
- Observe bounding box predictions.

Note: Full YOLO implementation requires specialized libraries (e.g., YOLOv5).

Expected Outcomes

- Understand CNN architecture and convolution operations.
- Build and train CNNs for multi-class image classification.
- Explain YOLO and residual learning concepts.
- Gain practical exposure to deep learning for computer vision.

Chapter 6

Recurrent Neural Networks

6.1 Overview

Recurrent Neural Networks (RNNs) are a class of neural networks designed for modeling sequential and temporal data. They are widely used in applications such as time-series forecasting, speech recognition, and natural language processing.

6.2 Basic Recurrent Unit (Elman Network)

The basic RNN, also known as the **Elman unit**, introduces feedback connections that allow information to persist over time.
Hidden State Update:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Output:

$$y_t = g(W_y h_t)$$

where h_t is the hidden state and x_t is the input at time t.

6.3 Training RNNs

RNNs are trained using **Backpropagation Through Time (BPTT)**.

- Unrolls the network across time steps.
- Computes gradients sequentially.
- Updates shared weights.

6.4 Advanced Recurrent Units

Long Short-Term Memory (LSTM)

LSTM networks address the limitations of basic RNNs by introducing memory cells and gates.

LSTM Gates:

- Forget Gate
- Input Gate
- Output Gate

Cell State Update:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Hidden State:

$$h_t = o_t \odot \tanh(C_t)$$

Gated Recurrent Unit (GRU)

GRU is a simplified version of LSTM with fewer gates.

GRU Gates:

- Update Gate
- Reset Gate

Advantages:

- Fewer parameters
- Faster training
- Comparable performance to LSTM

6.5 Vanishing Gradient Problem

Problem Description

In deep and recurrent networks, gradients can become very small during backpropagation, preventing effective learning of long-term dependencies.

Mitigation Techniques

- Using LSTM and GRU architectures
- Proper weight initialization
- Gradient clipping
- Batch normalization
- Shorter sequence lengths

Lab Exercise: Recurrent Neural Networks

Lab Objectives

After completing this lab, students will be able to:

- Build and train a basic RNN.
- Implement LSTM and GRU models.
- Understand sequence learning.
- Observe the effect of vanishing gradient mitigation.

Lab Steps

1. Simple RNN Implementation

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential([
    SimpleRNN(50, activation='tanh', input_shape=(10, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
```

2. LSTM Implementation

```
from tensorflow.keras.layers import LSTM

model = Sequential([
    LSTM(50, activation='tanh', input_shape=(10, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
```

3. GRU Implementation

```
from tensorflow.keras.layers import GRU

model = Sequential([
    GRU(50, activation='tanh', input_shape=(10, 1)),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse')
```

4. Gradient Clipping

```
from tensorflow.keras.optimizers import Adam

optimizer = Adam(clipnorm=1.0)

model.compile(optimizer=optimizer, loss='mse')
```

Expected Outcomes

- Understand the working of RNN, LSTM, and GRU.
- Observe how LSTM and GRU handle long-term dependencies.
- Apply techniques to mitigate vanishing gradients.
- Gain practical experience with sequence models.

Chapter 7

Transformers and Deep Generative Models

7.1 Transformers

Overview

The Transformer architecture is a deep learning model designed for sequence-to-sequence tasks. Unlike RNNs, Transformers rely entirely on attention mechanisms, enabling parallel processing and improved handling of long-range dependencies.

Encoder–Decoder Architecture

The Transformer consists of two main components:

- **Encoder**: Encodes the input sequence into a contextual representation.
- **Decoder**: Generates the output sequence using encoder outputs.

Each encoder and decoder block contains:

- Multi-head self-attention
- Feed-forward neural network
- Residual connections and normalization

Attention Mechanism

Attention allows the model to focus on relevant parts of the input sequence.
Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, and V are query, key, and value matrices.

Self-Attention and Multi-Head Attention

Self-Attention computes attention within the same sequence.
Multi-Head Attention runs multiple attention mechanisms in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W^O$$

Advantages:

- Captures diverse contextual relationships
- Improves model expressiveness

Positional Encoding

Since Transformers have no recurrence, positional information is added to embeddings.

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Pre-trained Language Models

Pre-trained models learn language representations from large corpora.
Examples:

- BERT – Bidirectional Encoder Representations from Transformers
- GPT – Generative Pre-trained Transformer

Advantages:

- Transfer learning
- Reduced training time
- High performance on NLP tasks

7.2 Autoencoders and Deep Generative Models

Autoencoders (AEs)

Autoencoders are neural networks trained to reconstruct their input.
Architecture:

- Encoder: Compresses input into latent representation
- Decoder: Reconstructs input from latent space

Reconstruction Loss:

$$L = \|X - \hat{X}\|^2$$

Types of Autoencoders

- **Undercomplete Autoencoder** – Latent dimension $<$ input
- **Regularized Autoencoder** – Adds constraints to avoid overfitting
- **Sparse Autoencoder** – Enforces sparsity in latent space
- **Denoising Autoencoder** – Learns to reconstruct clean input

Variational Autoencoder (VAE)

VAEs learn probabilistic latent representations.
Loss Function:

$$L = \text{Reconstruction Loss} + KL(q(z|x)\|p(z))$$

Applications: Image generation, anomaly detection.

Generative Adversarial Networks (GANs)

GANs consist of two competing networks:

- **Generator** – Produces fake samples
- **Discriminator** – Distinguishes real vs fake samples

Min–Max Objective:

$$\min_G \max_D \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

Conditional GAN (cGAN)

Conditional GANs generate data conditioned on class labels or attributes.
Advantages:

- Controlled data generation
- Improved sample quality

Lab Exercise: Transformers and Generative Models

Lab Objectives

After completing this lab, students will be able to:

- Understand Transformer architecture and attention mechanisms.
- Use pre-trained Transformer models.
- Implement autoencoders and GANs.

Lab Steps

1. Using a Pre-trained Transformer (BERT)

```python
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

inputs = tokenizer("Deep learning is powerful", return_tensors="pt")
outputs = model(**inputs)
```

2. Autoencoder Implementation

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

autoencoder = Sequential([
    Dense(64, activation='relu', input_shape=(100,)),
    Dense(32, activation='relu'),
    Dense(64, activation='relu'),
    Dense(100, activation='sigmoid')
])

autoencoder.compile(optimizer='adam', loss='mse')
```

3. GAN (Conceptual)

Task:

- Implement Generator and Discriminator networks.
- Train using adversarial loss.

Note: Full GAN training requires careful tuning and longer runtime.

Expected Outcomes

- Understand Transformer-based models and attention.
- Apply pre-trained NLP models.
- Implement autoencoders and generative models.
- Gain exposure to state-of-the-art deep learning techniques.

Chapter 8

Active Learning, Transfer Learning, and Applications of Deep Learning

8.1 Active Learning (AL)

Active Learning is a machine learning approach in which the model **actively selects the most informative samples** from unlabeled data for annotation. The goal is to achieve high performance using fewer labeled samples.
Motivation: - Labeling data is expensive and time-consuming - Not all data points contribute equally to learning
Active Learning Workflow:

1. Train an initial model with a small labeled dataset
2. Select informative unlabeled samples
3. Query labels from an oracle (human expert)
4. Retrain the model with newly labeled data

8.2 Sampling Techniques in Active Learning

Pool-Based Active Learning: - Samples are selected from a large pool of unlabeled data. - The most commonly used approach in practice.
Stream-Based Active Learning: - Data arrives sequentially. - The model decides whether to query a label for each instance.
Common Sampling Strategies:

- **Uncertainty Sampling:** Select samples with lowest prediction confidence.
- **Margin Sampling:** Select samples with smallest difference between top class probabilities.
- **Entropy-Based Sampling:** Select samples with highest entropy.

8.3 Transfer Learning (TL)

Transfer Learning leverages knowledge from a **source task** to improve performance on a **target task**. It is widely used in deep learning due to the availability of large pre-trained models.

Key Advantages:

- Faster training
- Reduced data requirements
- Improved generalization

Pre-Trained Models: - CNNs trained on ImageNet - Language models such as BERT, GPT

8.4 Transfer Learning Strategies

- **Feature Extraction:** Use pre-trained layers as fixed feature extractors.
- **Fine-Tuning:** Update pre-trained weights for the new task.
- **Multi-Task Learning:** Train a single model on multiple related tasks.
- **Zero-Shot Learning:** Predict unseen classes using semantic relationships.

8.5 Practical Applications of AL and TL

- Medical image diagnosis with limited labeled data
- Speech recognition systems
- Fraud detection
- Autonomous driving

8.6 Applications of Deep Learning Models

Deep learning has achieved state-of-the-art performance in many real-world applications.

Computer Vision:

- Image classification
- Object detection
- Face recognition

Natural Language Processing (NLP):

- Sentiment analysis
- Machine translation
- Question answering
- Text summarization

Exercises

1. Define Active Learning and explain why it is useful.
2. Compare pool-based and stream-based active learning.
3. Describe three sampling strategies used in Active Learning.
4. What is Transfer Learning? Explain its advantages.
5. Differentiate between feature extraction and fine-tuning.
6. Explain multi-task learning and zero-shot learning.
7. List real-world applications of Active Learning and Transfer Learning.
8. Describe two applications of deep learning in computer vision.
9. Explain the role of deep learning in Natural Language Processing.